Bharath Sai Chaitanya

20233102 B1

1) Assume that a system has a 32-bit virtual address with a 4-KB page size. Write a C program that is passed a virtual address on the command line and have it output the page number and offset for the given address.

```
#include <stdio.h>
#include <stdlib.h>
    int main (int argc, char * argv[]) {
        if (argc != 2) {
            printf ("usage %s <virtual address>\n",
        argv[0]);
            return 1;
        }
    unsigned int address = (unsigned int) strtoul (argv[1],
                NULL, 10);
unsigned int page-number = address >> 12;
    unsigned int offset = address & 0xFFF;

    printf (" The address %u contains:\n", address);
    printf (" Page number = %u\n", page-number);
    printf (" offset = %u\n", offset);

    return 0;
}
```

TERMINAL

```
PS C:\Users\ACER\Downloads\CODE> gcc test.c
>>
PS C:\Users\ACER\Downloads\CODE> .\a.exe 19986
>>
The address 19986 contains:
Page number = 4
Offset = 3602
PS C:\Users\ACER\Downloads\CODE>
```

Bharath Sai Chaitanya
20233102 B1

classmate
Date
Page

2) WAP that implements the FIFO and LRU page-replacement algorithms. First generate a random page reference string where page numbers range from 0 to 9. Apply the random page reference string to each algorithm and record the number of page faults incurred by each algorithm. Implement the replacement algorithms so that the number of page frame can vary from 1 to 7. Assume that demand paging is used.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

# define PAGE_RANGE 10
# define REF_STRING_LEN 20
    void generate_reference_string (int ref_str[]){
    for (int i=0; i<REF_STRING_LEN; i++){
        if (frame[i]==page) return 1;
        }
    return 0;
}

        int fifo (int ref_str[] int frame_size){
        int frame[frame_size], front=0, page_faults=0;
    for (int i=0; i<frame_size; i++) frame[i]=-1;
        for (int i=0; i<REF_STRING_LEN; i++){
            if (!is_in_frame (frame, frame_size, ref_str[i]))
    {       frame [front]= ref_str[i];
            front =(front+1)% frame_size;
            page_faults++;     } }
```

Bhanoth Sai Chaitanya
20233102 R1

```
return Page_faults ; }
int lru (int ref_str[] int frame_size){
  int frame[frame_size] used[frame_size]
  Page_faults = 0;
   for (int i= 0; i<frame_size; i++) {
       frame [i] = -1;
       used [i] = -1;      }
 for (int i=0; i<REF_STRING_LEN; i++){
    int found = 0;
   for (int j=0; j<frame_size; j++){
   if (frame[j] == ref_str[i]){
       used[j] = i;
       found = 1;
       break;        }
 } ~~if (!found)~~
    if (!found){
       int lru_index = 0;
       for (int j=1 ; j<frame_size ;j++) {
          if (used[j] < used[lru_index]){
             lru_index = j;
          }
 }
    frame [lru_index] = ref_str[i];
         used [lru_index] = i;
         Page_faults++;
       }
   }
     return page_faults ;
    }
```

Bharath Sai Chaitanya

2023 3102 B1

classmate

Date

Page

```
int main() {
    srand(time(NULL));
    int ref_str[REF_STRING_LEN];
      generate_reference_string(ref_str);

printf("Reference string: ");
    for (int i=0; i<REF_STRING_LEN; i++) {
        print("%d", ref str[i]);
    }
    printf("\n");

    for (int frames = 1; frames <= 7; frames++){
      int fifo_faults = fifo (ref_str, frames);
      int lru_faults = lru (ref_str, frames);
      printf("\nFor %d frames:\n", frames);
      printf("FIFO Page faults = %d \n", fifo_faults);
      printf("LRU Page faults = %d\n", lru_faults);
    }
return 0;
}
```

TERMINAL

```
PS C:\Users\ACER\Downloads\CODE> gcc test.c
PS C:\Users\ACER\Downloads\CODE> .\a
Page Reference String: 8 0 4 7 8 8 7 5 3 3 7 6 4 8 3 2 3 4 3 5

Frame Size: 1
FIFO Page Faults: 18
LRU Page Faults:  18

Frame Size: 2
FIFO Page Faults: 15
LRU Page Faults:  14

Frame Size: 3
FIFO Page Faults: 12
LRU Page Faults:  13

Frame Size: 4
FIFO Page Faults: 11
LRU Page Faults:  12

Frame Size: 5
FIFO Page Faults: 10
LRU Page Faults:  11

Frame Size: 6
FIFO Page Faults: 7
LRU Page Faults:  9

Frame Size: 7
FIFO Page Faults: 7
LRU Page Faults:  8

PS C:\Users\ACER\Downloads\CODE>
```

powersh...
powersh...
powersh...
powersh...
powershell
powershell
powershell
powershell
powershell
powershell
powershell